Manual técnico del sistema

Arquitectura, instalación, operación y recuperación de SGSPE

Equipo de SGSPE

Versión 3.0.0 · 2026-10-24: Edición basada en el código de la versión 3.0.0

Contenido

1. Introducción . . . **5**

1.1. Qué es SGSPE . . . 5

1.2. Alcance de este manual . . . 5

1.3. Convenciones . . . 6

2. Arquitectura . . . **7**

2.1. Contexto . . . 7

2.2. Contenedores y puertos . . . 7

2.3. Cómo se organiza el código del servidor . . . 9

2.4. Ruta de una escritura . . . 10

2.5. Decisiones de arquitectura . . . 10

3. Componentes y comportamiento interno . . . **12**

3.1. Paquetes del servidor . . . 12

3.2. Interfaz web (`sgspe-ui`) . . . 13

3.3. Trabajos programados . . . 13

3.4. Procesos por eventos . . . 14

3.5. Seguridad . . . 15

3.5.1. Autenticación . . . 15

3.5.2. Autorización . . . 15

3.5.3. Límites de confianza . . . 16

3.6. Auditoría y trazabilidad . . . 16

3.7. Validaciones y errores de la API . . . 16

4. Datos y base de datos . . . **18**

4.1. Motor y esquemas . . . 18

4.2. Cómo se crea el esquema . . . 18

4.3. Modelo de datos . . . 19

4.4. Archivos . . . 20

5. Configuración . . . **21**

5.1. Base de datos, seguridad y archivos . . . 21

5.2. Correo, mapas, GeoServer y PDF . . . 22

5.3. IA y trabajos . . . 23

5.4. Propiedades `sgspe.*` sin variable propia . . . 23

5.5. Perfil `prod` . . . 24

6. Instalación y arranque . . . **26**

6.1. Requisitos . . . 26

6.2. Paquetes de wasichai . . . 26

6.3. Entorno de desarrollo con Docker (vía oficial) . . . 27

6.4. Entorno de demostración sin Docker (verificado) . . . 28

6.4.1. Interfaz con paquetes compilados localmente . . . 28
6.5. Comprobaciones después de instalar . . . 29
7. Despliegue y actualización . . . 30
7.1. Artefactos . . . 30
7.2. Topología de referencia . . . 30
7.3. Comprobaciones de arranque (`ProdGuard`) . . . 31
7.4. Primer despliegue . . . 31
7.5. Actualizar a una versión nueva . . . 32
7.6. Volver a la versión anterior . . . 32
8. Operación y monitoreo . . . 33
8.1. Arrancar, detener y reiniciar . . . 33
8.2. Comprobar el estado . . . 33
8.3. Registros . . . 34
8.4. Tareas periódicas del operador . . . 34
8.5. Feriados para los días hábiles . . . 35
8.6. Comprobaciones después de un despliegue o una restauración . . . 35
9. Copias de seguridad y restauración . . . 36
9.1. Qué respaldar . . . 36
9.2. Respaldo lógico de la base . . . 36
9.3. Restaurar . . . 37
9.4. Prueba periódica . . . 38
10. Base de demostración reproducible . . . 39
10.1. Qué contiene . . . 39
10.1.1. Usuarios de demostración . . . 40
10.2. Requisitos . . . 40
10.3. Reconstruir desde cero . . . 41
10.4. Exportar . . . 42
10.5. Restaurar y verificar . . . 42
10.6. Reiniciar el entorno . . . 44
10.7. Mantener los conjuntos . . . 44
11. Diagnóstico de fallos frecuentes . . . 45
11.1. Compilación y arranque . . . 45
11.2. Uso del portal . . . 45
11.3. Datos y modelo . . . 47
12. Recuperación ante desastres . . . 48
12.1. Lista de comprobación de recuperación . . . 49
13. Pruebas y calidad . . . 50
13.1. Niveles de prueba . . . 50
13.2. Integración continua . . . 51

13.3. Verificaciones hechas para estos manuales . . . 51
14. Mantenimiento de esta documentación . . . 52
14.1. Estructura . . . 52
14.2. Herramientas . . . 52
14.3. Comandos . . . 53
14.4. Actualizar las capturas . . . 53
14.5. Validación . . . 54
14.6. Cuándo actualizar . . . 54

Presentación

Este manual permite a un ingeniero que no conoce SGSPE comprender cómo está construido, instalarlo, configurarlo, operarlo, actualizarlo y recuperarlo ante una incidencia. Describe el sistema tal como está en el código de la versión 3.0.0: cada afirmación remite al archivo que la respalda, y lo que no pudo verificarse se señala como tal.

Público: administradores técnicos, operadores de plataforma, DBA y desarrolladores. El uso funcional del portal se explica en el Manual de usuario; la administración funcional (usuarios, roles, modelo), en el Manual del administrador; el detalle de objetos, permisos, workflows y API, en la Referencia técnica.

1. Introducción

1.1. Qué es SGSPE

SGSPE (*Sistema de Gestión Social para Proyectos Extractivos*) es una aplicación web para que el equipo social de un proyecto minero, de hidrocarburos o de energía gestione en un solo lugar el territorio y las áreas de influencia, los grupos de interés y el relacionamiento con ellos, los compromisos, las quejas, consultas y solicitudes (QCS), los riesgos y las alertas, la inversión social, los tableros de indicadores y los entregables DOCX y PDF.

Se compone de dos repositorios:

Repositorio	Contenido	Tecnología
`sgspe` (este)	Servidor (API, reglas, trabajos), modelo como código (`model/`), documentación de diseño (`docs/`) y estos manuales	Kotlin 2.4, Spring Boot 4.1 WebFlux, JDK 25, wasichai 1.3.0
`sgspe-ui`	Interfaz web: el portal en `/` y la administración de la plataforma en `/admin`	React 19, Vite 8, TypeScript, paquetes `@wasichai/*` 1.0.0

SGSPE se construye sobre **wasichai**, una plataforma de metadatos: los objetos de negocio (proyecto, grupo de interés, compromiso…) no son clases ni tablas escritas a mano, sino definiciones en `model/model.json` que la plataforma convierte en tablas, API REST genérica, permisos, workflows, formularios y capas SIG. SGSPE agrega encima las reglas que la plataforma no trae (guardas de escritura, comandos de varios registros, trabajos programados, tableros, entregables e IA) en los paquetes `sgspe.*`.

1.2. Alcance de este manual

Capítulo	Para qué sirve
Capítulo 2	Entender los componentes, cómo se comunican y por qué están así.
Capítulo 3	Conocer el código: paquetes, trabajos, eventos, seguridad y auditoría.
Capítulo 4	Saber dónde viven los datos y cómo se crean las tablas.
Capítulo 5	Consultar cada variable de entorno y propiedad.
Capítulo 6	Instalar un entorno de desarrollo o de demostración paso a paso.
Capítulo 7	Desplegar en producción y actualizar versiones.

Capítulo	Para qué sirve
Capítulo 8	Arrancar, detener, vigilar y leer los registros.
Capítulo 9 y Capítulo 10	Respaldar, restaurar y reconstruir la base de demostración.
Capítulo 11 y Capítulo 12	Diagnosticar fallos y recuperarse de un desastre.
Capítulo 13	Ejecutar las pruebas y entender la CI.
Capítulo 14	Mantener y regenerar esta documentación.

1.3. Convenciones

- Los comandos se muestran con el directorio desde el que se ejecutan. `$` indica el intérprete de órdenes; no se copia.
- Los valores entre `<` y `>` (por ejemplo `<slug>`) se reemplazan por los propios.
- Nunca se escriben contraseñas ni claves reales: los ejemplos usan los valores de desarrollo de `desarrollo/example.env`, que solo valen en una máquina local, o `…` donde el valor debe llegar por una variable de entorno.
- Los nombres técnicos (objetos, campos, estados, roles, acciones) están en inglés y su texto visible en español, como manda el ADR-0009 del proyecto.
- Las referencias `archivo.kt:línea` remiten al código del repositorio en la versión 3.0.0.



Fuente de verdad

El diseño de SGSPE vive en `docs/` (análisis de requisitos, arquitectura, ADR). Este manual describe lo implementado y remite a esos documentos para el porqué; si el diseño y el código difieren, manda el código para saber qué hace hoy el sistema, y el diseño para saber qué se pretende.

2. Arquitectura

2.1. Contexto

SGSPE es un solo sistema por organización cliente (*tenant*). Lo usan el equipo social y de proyecto, con alguno de los 15 roles declarados en `model/roles.json`, y la administración de la organización, con el rol `ADMIN` de wasichai. Los actores del territorio (personas, comunidades, organizaciones) se registran como datos y nunca inician sesión.

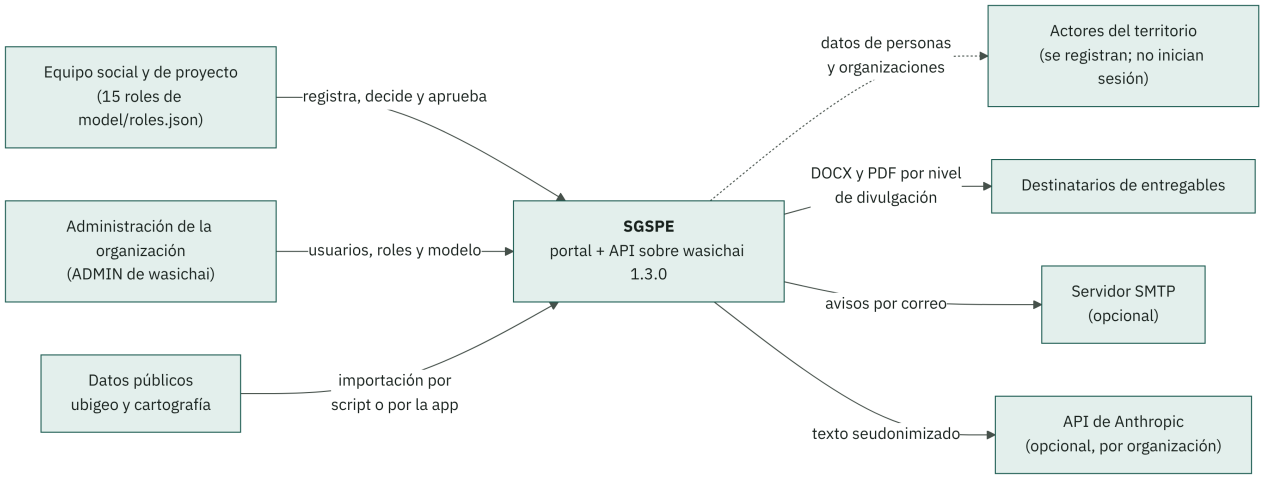


Figura 1. Contexto del sistema (C4, nivel 1)

Las dos salidas externas son opcionales y se encienden por configuración: el correo de los avisos (`WASICHAI_NOTIFICATIONS_EMAIL_ENABLED`) y la IA (`SGSPE_AI_ENABLED_ORGANIZATIONS` más `ANTHROPIC_API_KEY`), que solo recibe texto seudonimizado.

2.2. Contenedores y puertos

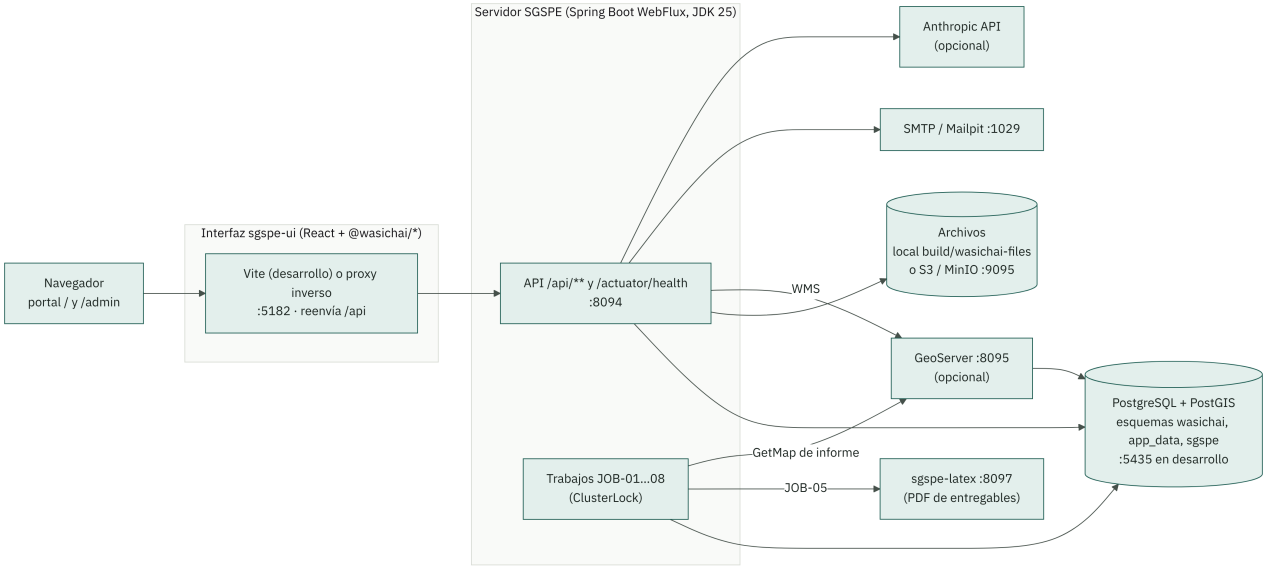


Figura 2. Contenedores, puertos y dependencias

Tabla 1. Puertos de los servicios (desarrollo)

Servicio	Puerto	Notas
Interfaz `sgspe-ui` (Vite `dev` o `preview`)	5182	Reenvía `/api` al servidor (`WASICHAI_API_URL`, por defecto `http://localhost:8094`).
Servidor SGSPE	8094	`server.port` en `application.yml`. Sirve `/api/` y `/actuator/health/` en el mismo puerto.
PostgreSQL + PostGIS	5435	`compose.yml` publica el 5432 del contenedor en `127.0.0.1:5435`.
MinIO (API / consola)	9095 / 9096	Solo con `WASICHAI_FILES_STORE=s3`; en producción, cualquier S3.
Mailpit (SMTP / web)	1029 / 8029	Recibe los correos de prueba sin entregarlos.
GeoServer	8095	Opcional; nunca público: el navegador lo ve por el proxy WMS del servidor.
`sgspe-latex`	8097	Compone el PDF de los entregables (JOB-05). Solo lo llama el servidor.

En producción la topología de referencia (`docs/architecture.md` §10, ADR-0026) es **un solo origen** detrás de un proxy inverso con TLS: el proxy sirve los estáticos de `sgspe-ui` con *fallback* de SPA (incluido `/admin/*`) y reenvía `/api/` y `/actuator/health/` al servidor. PostgreSQL, GeoServer, `sgspe-latex` y el almacenamiento S3 quedan en una red interna. El destino concreto de producción no está definido en el repositorio (GAP-022).

2.3. Cómo se organiza el código del servidor

El servidor es una sola aplicación Spring Boot (`sgspe.SgspeApplication`) que incluye los *starters* de wasi-chai (core, views, forms, pages, gis, workflow, automation, documents, files, notifications y agent; `build.gradle.kts`) y los paquetes propios `sgspe.*`. Las dependencias entre paquetes son las reales del código (imports de `src/main/kotlin`) y las vigila `ArchitectureTest`.

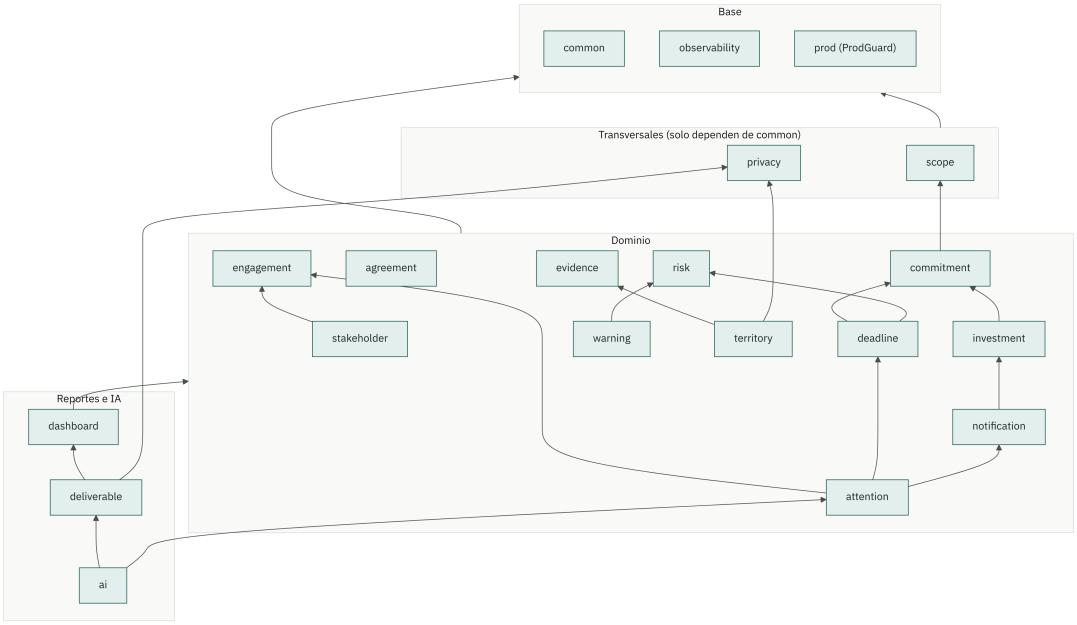


Figura 3. Dependencias entre los paquetes `sgspe.*`

Reglas de arquitectura que fallan la compilación de pruebas si se rompen (`src/test/kotlin/sgspe/ArchitectureTest.kt`):

- ningún código bajo el paquete `wasichai` ni uso de sus API internas (`RecordStore`, `AuditService`, `ObjectSchemaManager`…);
- ningún nombre de rol en `src/main/kotlin`: el código pregunta por **acciones** (`APPROVE`, `EXPORT`…), nunca por roles;
- `sgspe.scope` y `sgspe.privacy` solo dependen de `sgspe.common`; los paquetes de dominio no dependen de tableros, entregables ni IA;
- solo `sgspe.ai` usa el módulo de agentes y Embabel; solo `KnowledgeChunks` toca la tabla `knowledge_chunk`; solo `SpatialGate` (y los escritores de territorio) ejecuta SQL propio.

2.4. Ruta de una escritura

Hay dos caminos para escribir:

1. **API genérica de wasichai** (`/api/objects/{objeto}/records`): la usan los formularios del portal y de `/admin`. La plataforma valida el token, aplica el permiso de rol, objeto y campo, el alcance de lectura, y llama a las guardas de SGSPE antes de guardar. La escritura y su fila de auditoría salen en la misma llamada.
2. **Comandos de SGSPE** (`/api/sgspe/**`): operaciones que escriben varios registros a la vez (registrar una entrega, aceptar una solicitud, emitir un entregable…). Corren en una transacción, con candado consultivo de PostgreSQL y las cabeceras `Idempotency-Key` (un reenvío devuelve la respuesta guardada), `If-Match` (rechaza con 412 si otra persona cambió el registro) y `X-Change-Reason` (el motivo queda en la auditoría).

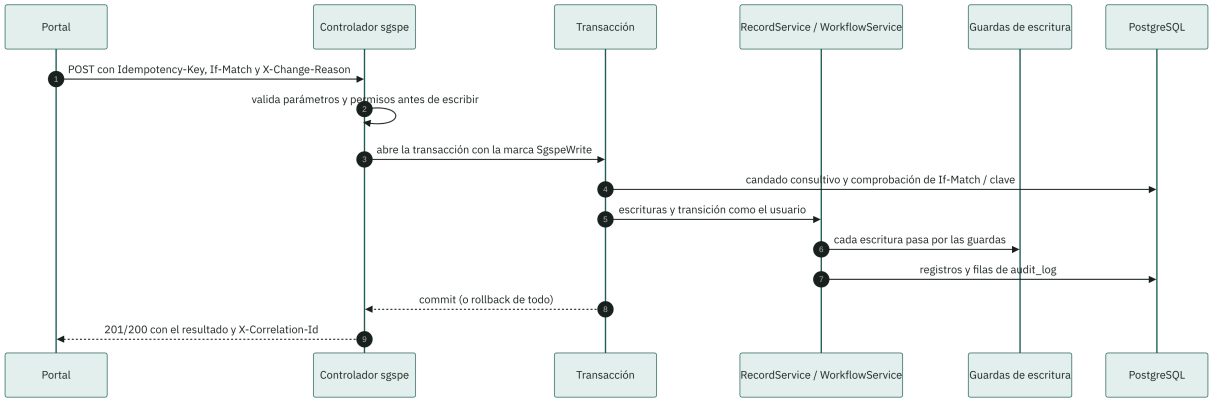


Figura 4. Secuencia de un comando de `/api/sgspe/**`

Toda respuesta lleva `X-Correlation-Id`, que también aparece en la línea de acceso y en la auditoría: es la forma de seguir una petición de punta a punta (ver Sección 8.3).

2.5. Decisiones de arquitectura

Las decisiones están registradas como ADR en `docs/adr/`. Las que más pesan en la operación:

ADR	Decisión y consecuencia operativa
0001	Construir sobre wasichai: el esquema de base de datos lo crea la plataforma a partir del modelo.
0002	Geometrías en SRID 4326; cada proyecto tiene un CRS de trabajo para medir y exportar.
0006, 0029	Autorización por roles y **alcance por proyecto**: una persona no administradora solo lee los proyectos donde tiene una asignación vigente.

ADR	Decisión y consecuencia operativa
0007	El modelo como código en `model/`: `/admin` no es fuente de verdad; volver a aplicar `model/` deshace los cambios manuales.
0011	Auditoría sobre el `audit_log` de wasichai (solo inserción).
0012	Estados temporales (vencido, por vencer…) materializados por trabajos programados.
0014	Evidencias y entregables en almacenamiento de objetos (S3 en producción).
0020	JWT de wasichai endurecido: duración corta y revocación en producción.
0025, 0027	Concurrencia: `Idempotency-Key` e `If-Match` en la API del portal.
0026	Topología de un solo origen; puertos y observabilidad.
0030	PDF de imprenta con LuaLaTeX (`sgspe-latex`); reemplazó a Gotenberg el 2026-10-19.

3. Componentes y comportamiento interno

3.1. Paquetes del servidor

Paquete	Responsabilidad (evidencia en `src/main/kotlin/sgspe/<paquete>/`)
`common`	Base compartida: lectura y escritura «como quien llama» (`Records`), permisos por acción (`Permissions`), idempotencia (`Idempotency`, tabla `sgspe.idempotency_key`), `If-Match` (`Preconditions`), consulta en seco de transiciones (`TransitionChecks`), días hábiles y feriados, códigos propuestos, JOB-02 y JOB-08.
`observability`	`X-Correlation-Id` y la línea de acceso `sgspe.access`.
`prod`	`ProdGuard`: impide arrancar con el perfil `prod` si la configuración es insegura (Sección 7.3).
`scope`	Alcance por proyecto: `ProjectReadScope` (lectura), `ProjectScopeGuard` (escritura con asignación vigente), `EditAnyGuard` (solo lo propio sin `EDIT_ANY`), QCS confidenciales.
`privacy`	Exportaciones XLSX con nivel de divulgación, celdas pequeñas y `export_log`; modelo del informe.
`stakeholder`	Evaluaciones (índices de interés y poder, grupo de influencia), prioridad, WF-B y JOB-03.
`engagement`	WF-C y conteos de asistentes.
`territory`	CRS, áreas de influencia, importaciones con GDAL, capas LYR-01 a LYR-24, consultas espaciales por `SpatialGate`, proxy WMS, exportaciones geográficas y calidad de datos.
`commitment, agreement`	Guardas de compromisos (D1–D11) y de acuerdos; entregas, reprogramaciones y candidatos.
`evidence`	Evidencias sobre wasichai-files; descarga con `DOWNLOAD_RESTRICTED`.
`investment`	Aportes y programas sociales: topes de apoyo, aprobaciones (P1–P9, G1–G4), totales.
`attention`	QCS: plazos, reclasificación, aceptación, derivación, cola unificada y reporte.
`risk, warning`	Riesgos y su revisión; reglas de alerta temprana (JOB-04).
`deadline, notification`	JOB-01 (plazos y escalamientos) y avisos sobre wasichai-notifications.
`dashboard`	KPI y tableros (`stakeholders`, `engagement`, `commitments`, `attention`, `risks`, `investment`, `territory`, `executive`, `project`, `monitoring`); cierre de periodo de indicadores.

Paquete	Responsabilidad (evidencia en `src/main/kotlin/sgspe/<paquete>/`)
`deliverable`	Entregables T1 y T2: JOB-05, DOCX, PDF con LuaLaTeX, mapas de informe, plantillas y emisión.
`ai`	IA opcional: asistente, preguntas a documentos (RAG), duplicados, casos con sugerencias, seudonimización (JOB-07).

3.2. Interfaz web (`sgspe-ui`)

Una sola compilación de Vite sirve dos aplicaciones (`src/main.tsx`): si la ruta empieza por `/admin` se monta la administración de wasichai (`AdminApp`, módulos de datos, GIS, constructor, automatización y administración); si no, el portal de SGSPE (`PortalApp`). Ambas comparten la sesión: el token se guarda en `localStorage` con el prefijo `sgspe` y cada llamada lo envía como `Authorization: Bearer`.

Variable de compilación	Efecto (`vite.config.ts`)
`WASICHAI_API_URL`	Destino del proxy `/api` de `vite dev` y `vite preview` (por defecto `http://localhost:8094`).
`SGSPE_ROL_JEFATURA`	Rol que el portal muestra como «Jefatura» del proyecto (por defecto `PROJECT_MANAGER`).
`SGSPE_MAPA_BASE`	Orígenes del mapa base que se agregan a la CSP. El estilo real llega del servidor (`/api/sgspe/map/settings`).

La CSP del portal se envía en modo de solo informe (`Content-Security-Policy-Report-Only`) desde Vite en desarrollo; en producción la debe enviar el proxy. `yarn csp` (en `sgspe-ui`) imprime la política para copiarla en el proxy.

3.3. Trabajos programados

Todos los trabajos se apagan con `SGSPE_JOBS_ENABLED=false` (`sgspe.jobs.enabled`), toman un candado de clúster (corren en una sola réplica) y usan cron de Spring (seis campos, el primero son segundos) en la zona `America/Lima`.

Tabla 2. Trabajos programados (`src/main/resources/application.yml`)

Id	Clase	Cuándo (por defecto)	Qué hace
JOB-01	`deadline.DeadlineJob`	`0 5 * * * *`	Cada hora: escribe el estado de plazo de compromisos, tareas, quejas y consultas y la vigencia de acuerdos; crea tareas de escalamiento; mide la revisión de riesgos. Nunca aplica transiciones.
JOB-02	`common.DerivedFieldsJob`	`0 15 * * * * y 0 45 3 * * *`	Recalcula los campos derivados que un listener no pudo escribir (incremental cada hora; completo a las 03:45).
JOB-03	`stakeholder.StakeholderFollowUpJob`	`0 30 4 * * *`	Último contacto de los grupos de interés; tareas de evaluación periódica y de inactividad.
JOB-04	`warning.WarningRulesJob`	`0 25 * * * *`	Evalúa las reglas de alerta temprana y crea alertas en «Detectada». Nunca confirma.
JOB-05	`deliverable.DeliverableGenerationJob`	`cada 5 s`	Genera los borradores pedidos de los entregables (DOCX, mapas, PDF con `sgspe-latex`); retoma los colgados tras 15 min.
JOB-07	`ai.KnowledgeIndexJob`	`0 */30 * * * * y 0 15 3 * * *`	Fragmenta el texto de las evidencias para las preguntas a documentos; solo en organizaciones con IA.
JOB-08	`common.RetentionJob`	`0 15 4 * * *`	Borra las claves de idempotencia vencidas (24 h) y las versiones de derivados de más de 7 días.

JOB-06 no existe: se retiró en la fase 3 junto con las tablas propias de notificaciones. Las propiedades de cada cron están en Sección 5.4.

3.4. Procesos por eventos

Además de los trabajos, 17 *listeners* de cambios de registro reaccionan dentro de la misma escritura (por ejemplo, `QcsDueDatesListener` calcula los plazos de una queja al registrarla, `ContributionTotalsListener` suma los ítems de un aporte y `RequestClosureListener` cierra una solicitud cuando se cierra su compromiso). Las **automatizaciones** declaradas en `model/automations/` (AUT-01, 02, 05, 09, 10, 11 y 12) las ejecuta wasichai: crean tareas o generan documentos al entrar en un estado; su lista está en la Referencia técnica.

3.5. Seguridad

3.5.1. Autenticación

- `POST /api/auth/login` recibe correo y contraseña y devuelve un JWT HS256 firmado con `WASICHAI_JWT_SECRET` (claims `sub`, `org`, `email`, `roles`). La organización sale del token, nunca del cuerpo de la petición.
- Duración: 8 h por defecto en wasichai; **4 h** con el perfil `prod` (`wasichai.security.jwt.ttl: PT4H`), con revocación activa y límite de intentos de inicio de sesión (5 por ventana de 15 min, 20 por cuenta).
- Contraseñas en `prod`: mínimo 12 caracteres con mayúscula, minúscula, dígito y símbolo, distinta del correo.
- Son públicas solo `/api/auth/login`, `/api/auth/token` (cuentas de servicio), `/api/health` y `/actuator/health/**`.
- «Cerrar sesión» en el portal borra el token del navegador; `POST /api/auth/logout` (revoca todos los tokens del usuario) existe en la API.

3.5.2. Autorización

La autorización tiene cuatro capas, en este orden:

1. **Permiso de rol por objeto y acción** (`model/roles.json` aplicado con `apply_roles.py`): `READ`, `CREATE`, `UPDATE`, `DELETE` y acciones declaradas (`APPROVE`, `VERIFY`, `EXPORT`, `ISSUE`, `CONFIRM_ALERT`, `DOWNLOAD_RESTRICTED`, `EDIT_ANY`, `READ_ALL_PROJECTS`, `READ_CONFIDENTIAL`…).
2. **Permiso de campo**: los campos restringidos (postura, documento de identidad, datos de contacto, titulares de predios…) solo los ven los roles lectores declarados.
3. **Alcance por proyecto**: quien no es administrador ni tiene `READ_ALL_PROJECTS` solo lee los registros de los proyectos donde tiene una `project_assignment` vigente, y solo escribe en ellos. Las QCS confidenciales exigen `READ_CONFIDENTIAL`.
4. **Guardas de negocio**: reglas de cada transición (por ejemplo, quien aprueba un compromiso no puede ser quien lo registró; quien verifica no puede ser el responsable).

El detalle por rol está en la Referencia técnica (matriz de permisos generada de `roles.json`).

3.5.3. Límites de confianza

- GeoServer nunca es público: el navegador pide las capas al proxy WMS autenticado del servidor, que solo deja pasar capas de la lista blanca del tenant (`/api/sgspe/territory/wms`).
- `sgspe-latex` solo lo llama JOB-05; no recibe URL ni opciones del cliente y no tiene red de salida en producción.
- La IA recibe solo texto seudonimizado; la clave del proveedor llega únicamente por `ANTHROPIC_API_KEY` y los registros nunca llevan preguntas, prompts ni respuestas.
- La ruta genérica del agente de wasichai (`/api/agent/ask`) está cerrada con 403 (`AgentHttpGate`).

3.6. Auditoría y trazabilidad

- `wasichai.audit_log` es **solo de inserción** (protegido con *triggers*): una fila por cada alta, cambio, borrado o emisión, con el estado anterior y el nuevo en `jsonb`, el usuario, el motivo (`X-Change-Reason`), el `correlation_id` y el origen (`api`, `platform`, `automation:…`, `job:…`).
- Se consulta por la API en `GET /api/audit` y `GET /api/objects/{objeto}/records/{id}/history`, en `/admin` (Auditoría) y en la pestaña «Actividad» de las fichas del portal.
- SGSPE agrega `export_log` (una fila por exportación, emisión o descarga) y `ai_interaction` (una por cada llamada al modelo de IA).

3.7. Validaciones y errores de la API

Los errores siguen el formato *problem+json* con un `code` estable. Los más frecuentes:

Estado	Significado
400	Datos inválidos (`VALIDATION`, `FILE_TOO_LARGE`, `OUT_OF_RANGE`, `TOO_MANY_RECORDS`…).
401	Sin token o token vencido.
403	Falta el permiso o la acción; el cuerpo nombra el permiso que falta.
404	El registro no existe o no está en el alcance de quien llama.
409	Conflicto de estado, registro de solo inserción o petición idempotente en curso (con `Retry-After`).
412	`If-Match` no coincide: otra persona cambió el registro.
422	Parámetro de consulta no declarado o regla de negocio incumplida (`FORMAT_UNAVAILABLE` sin GDAL, etc.).

Estado	Significado
503	Servicio opcional no disponible (`AI_UNAVAILABLE`, `AI_BUSY`).

4. Datos y base de datos

4.1. Motor y esquemas

SGSPE usa **PostgreSQL con PostGIS** (la imagen de referencia es `postgis/postgis:18-3.6`, `compose.yml`). La base tiene tres esquemas propios:

Esquema	Contenido
`wasichai`	Metadatos y tablas de la plataforma: organizaciones, usuarios, roles, permisos, objetos y campos del modelo, relaciones, workflows, automatizaciones, documentos, archivos (`stored_files`), notificaciones y la auditoría (`audit_log`). Cada módulo tiene su historia de migraciones `flyway_history_<módulo>`.
`app_data`	Los registros de negocio: una tabla física por objeto del modelo y organización, llamada `<objeto>_<8 primeros caracteres hexadecimales del id de la organización>` (por ejemplo `commitment_1a2b3c4d`), más las tablas de unión de las relaciones varios a varios.
`sgspe`	Tablas fijas de SGSPE: `idempotency_key`, `derived_version` y `knowledge_chunk`.

La extensión `postgis` vive en `public`. Las geometrías se guardan en SRID 4326 (ADR-0002).

4.2. Cómo se crea el esquema

El esquema se crea en dos etapas, ambas automáticas e idempotentes:

1. **Al arrancar el servidor**, `WasichaiMigrations` aplica con Flyway las migraciones de cada módulo en orden: core (0), semilla de desarrollo (10, solo con `WASICHAI_SEED_DEV=true`), módulos de wasichai (100+) y los de SGSPE: `sgspe_common` (200: `V1__idempotency_key.sql`, `V2__derived_version.sql`) y `sgspe_ai` (201: `V1__knowledge_chunk.sql`), en `src/main/resources/db/sgspe/`. `wasichai.database.migrate=false` las desactiva.
2. **Al aprovisionar una organización**, `model/apply.py` publica el modelo por la API de metadatos: la plataforma crea en `app_data` una tabla por objeto con sus columnas, índices y restricciones de unicidad, y agrega la columna de estado cuando `apply.py --workflows` publica el workflow del objeto.

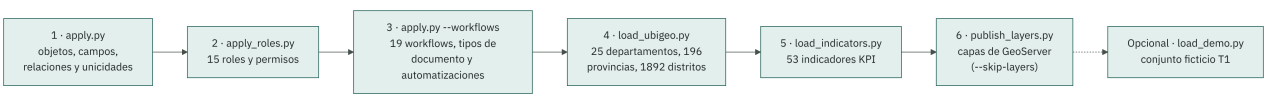


Figura 5. Orden de aprovisionamiento de una organización

> Como el nombre de cada tabla física incluye el id de la organización, **un volcado de datos solo se puede restaurar junto con su esquema** (el volcado completo de Capítulo 9), nunca

sobre un esquema creado de nuevo con `apply.py`, que tendría otro id de organización.

4.3. Modelo de datos

El modelo declara 90 objetos, 79 listas de opciones y 293 relaciones en `model/model.json`. El núcleo:

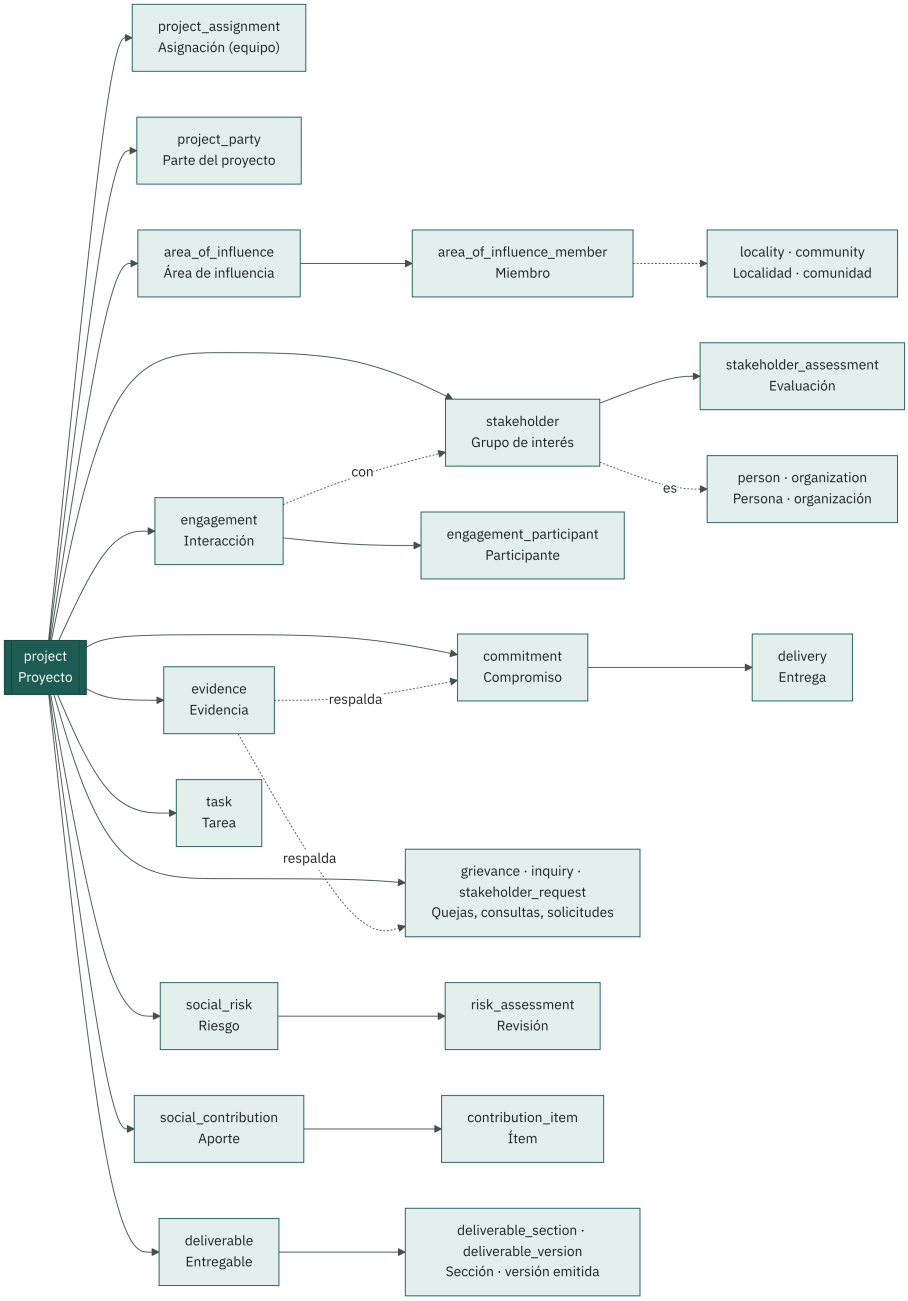


Figura 6. Núcleo del modelo de datos

Reglas que conviene conocer antes de tocar datos directamente en la base (lo que **no** se debe hacer):

- **No escribir en `app_data` con SQL**: se saltan los permisos, las guardas, los campos derivados, la auditoría y los listeners. Toda corrección de datos va por la API o el portal, con motivo.

- Los campos **derivados** (por ejemplo `deadline_status`, `executed_amount`, los índices de un grupo de interés) los calcula el servidor; la API rechaza que se envíen.
- Los objetos **de solo inserción** (`stakeholder_assessment`, `risk_assessment`, `commitment_reschedule`, `indicator_measurement`, `deliverable_version`, `export_log`, `ai_interaction`…) no se modifican: responden 409.
- `audit_log` está protegido por *triggers* contra modificaciones y borrados.

El catálogo completo de objetos, campos, relaciones y marcas está en la Referencia técnica, generado de `model.json`.

4.4. Archivos

Los archivos (evidencias, borradores y versiones emitidas de entregables, plantillas) los guarda **wasichai-files**: el descriptor va en `wasichai.stored_files` y los bytes en el almacenamiento configurado:

- `WASICHAI_FILES_STORE=local`: en la carpeta `WASICHAI_FILES_LOCAL_PATH` (por defecto `build/wasichai-files`). Solo para desarrollo y demostración.
- `WASICHAI_FILES_STORE=s3`: en el bucket `WASICHAI_FILES_S3_BUCKET` (MinIO en desarrollo; obligatorio en producción).



Un respaldo de SGSPE son **dos** cosas que deben corresponderse: la base de datos y el almacenamiento de archivos. Restaurar solo la base deja descriptores de archivos sin bytes.

5. Configuración

La configuración del servidor está en `src/main/resources/application.yml` (valores por defecto, pensados para desarrollo) y `application-prod.yml` (perfil `prod`). Cada propiedad se puede fijar con su variable de entorno: Spring traduce `wasichai.database.host` a `WASICHAI_DATABASE_HOST`, pero el proyecto declara además variables con nombre propio (columna *Variable*), que son las que se documentan y se usan. La interfaz tiene sus propias variables de compilación (Capítulo 3).



Las contraseñas y claves de esta tabla son las de **desarrollo** de `develop/example.env` y `compose.yml`. En cualquier entorno compartido o de producción se reemplazan por secretos propios que se inyectan como variables de entorno desde el gestor de secretos del despliegue. Nunca se escriben en un archivo versionado, y la clave del proveedor de IA tampoco en `develop/.env`.

5.1. Base de datos, seguridad y archivos

Variable	Por defecto (desarrollo)	Efecto
`WASICHAI_DB_HOST`	`localhost`	Servidor PostgreSQL. En `prod`, sin valor por defecto: sin la variable, no arranca.
`WASICHAI_DB_PORT`	`5435` (`prod`: `5432`)	Puerto.
`WASICHAI_DB_NAME`	`sgspe`	Base. En `prod`, obligatoria.
`WASICHAI_DB_USERNAME` / `_PASSWORD`	`sgspe` / `sgspe`	Credenciales. En `prod`, obligatorias. **Secreto.**
`WASICHAI_JWT_SECRET`	`valor de desarrollo`	Firma HS256 de los tokens, 32 bytes o más. En `prod`, obligatorio y distinto del de desarrollo. **Secreto.**
`WASICHAI_SEED_DEV`	`true`	Crea `admin@wasichai.local` / `admin`. En `prod` vale `false` y `true` impide el arranque.
`WASICHAI_FILES_STORE`	`local`	`local` o `s3`. En `prod`, `s3` obligatorio.
`WASICHAI_FILES_LOCAL_PATH`	`build/wasichai-files`	Carpeta de los archivos con `local`.
`WASICHAI_FILES_S3_BUCKET`	`sgspe-evidencias`	Bucket de los archivos con `s3`.
`WASICHAI_FILES_S3_ENDPOINT`	`http://localhost:9095`	Punto de acceso S3 (MinIO en desarrollo).
`WASICHAI_FILES_S3_REGION`	`us-east-1`	Región.

Variable	Por defecto (desarrollo)	Efecto
WASICHAI_FILES_S3_PATH_STYLE_ACCESS	true (prod: false)	Rutas al estilo de MinIO.
WASICHAI_FILES_S3_ACCESS_KEY / _SECRET_KEY	sin valor	Credenciales S3; sin ninguna, rige la cadena del SDK (rol IAM). **Secreto.**

5.2. Correo, mapas, GeoServer y PDF

Variable	Por defecto	Efecto
WASICHAI_NOTIFICATIONS_EMAIL_ENABLED	false	Envía los avisos también por correo.
WASICHAI_NOTIFICATIONS_EMAIL_FROM	avisos@sgspe.test	Remitente. En prod no puede ser @sgspe.test.
SPRING_MAIL_HOST / SPRING_MAIL_PORT	localhost / 1029 (prod: sin host / 587)	Servidor SMTP.
SGSPE_MAP_BASE_STYLE_URL	https://demotiles.maplibre.org/style.json (prod: vacío)	Estilo del mapa base; vacío, el portal muestra «Sin mapa base».
WASICHAI_GIS_GEOSERVER_ENABLED	false	Integra GeoServer; apagado, las capas se listan sin publicar.
WASICHAI_GIS_GEOSERVER_URL	http://localhost:8095/geoserver	URL de la REST de GeoServer.
WASICHAI_GIS_GEOSERVER_USERNAME / _PASSWORD	sgspe / sgspe-geoserver	Credenciales de GeoServer. **Secreto.**
WASICHAI_GIS_GEOSERVER_DATASTORE_*	postgis, 5432, sgspe, sgspe_geoserver	Cómo llega GeoServer a PostGIS, con un rol de solo lectura.
SGSPE_DELIVERABLE_LATEX_URL	http://localhost:8097 (prod: vacío)	sgspe-latex; sin él, los borradores quedan FALLIDO con PDF_CONVERTER_UNAVAILABLE.
SGSPE_GDAL_PATH	vacío (usa el PATH)	Carpeta de ogrinfo y ogr2ogr 3.8+; sin GDAL, Shapefile, KML, GPKG, FileGDB y GPX responden 422 FORMAT_UNAVAILABLE.

5.3. IA y trabajos

Variable	Por defecto	Efecto
`SGSPE_AI_ENABLED_ORGANIZATIONS`	`vacío`	*Slugs* de las organizaciones con IA, separados por comas. Vacío: IA apagada para todas.
`ANTHROPIC_API_KEY`	`sin valor`	Clave del proveedor. Solo por variable de entorno, nunca en un archivo. Sin ella, la IA responde 503 `AI_UNAVAILABLE` y el resto funciona. **Secreto.**
`SGSPE_JOBS_ENABLED`	`true`	Apaga todos los trabajos programados (útil en una réplica de solo lectura o al restaurar).
`SPRING_PROFILES_ACTIVE`	`sin valor`	`prod` activa `application-prod.yml` y `ProdGuard`.
`SERVER_PORT`	`8094`	Puerto HTTP (variable estándar de Spring Boot).

5.4. Propiedades `sgspe.*` sin variable propia

Se cambian en `application.yml` o con la variable equivalente de Spring (por ejemplo `SGSPE_EXPORTS_MAX_ROWS`).

Propiedad	Por defecto	Efecto
`sgspe.jobs.deadline.cron`	`0 5 * * * *`	JOB-01.
`sgspe.jobs.derived-fields.hourly-cron / daily-cron`	`0 15 * * * * / 0 45 3 * * *`	JOB-02.
`sgspe.jobs.stakeholder-follow-up.cron`	`0 30 4 * * *`	JOB-03.
`sgspe.jobs.warning-rules.cron`	`0 25 * * * *`	JOB-04.
`sgspe.deliverable.generation.poll / stale-after`	`5s / 15m`	JOB-05.
`sgspe.jobs.knowledge-index.cron / daily-cron`	`0 */30 * * * * / 0 15 3 * * *`	JOB-07.
`sgspe.jobs.retention.cron`	`0 15 4 * * *`	JOB-08.

Propiedad	Por defecto	Efecto
`sgspe.exports.max-rows`	`20000`	Máximo de filas de una exportación; más: 400 `TOO_MANY_RECORDS`.
`sgspe.territory.import.max-bytes / max-rows / timeout`	`20 MB / 10000 / 60s`	Límites de una importación geográfica.
`sgspe.territory.layers.max-features`	`5000`	Máximo de geometrías por capa.
`sgspe.territory.queries.*`	`buffer 1–20000 m; distancia 0,1–50 km`	Rangos de las consultas espaciales; fuera de rango: 400 `OUT_OF_RANGE`.
`sgspe.deliverable.latex.timeout`	`180s`	Plazo de una composición de PDF.
`sgspe.calendar.holidays`	`2026 y 2027`	Feriados nacionales para los días hábiles de las QCS (ver Sección 8.5).
`sgspe.dashboards.executive.max-projects`	`25`	Proyectos máximos de la revisión ejecutiva.
`sgspe.ai.limits.max-concurrent-runs`	`3`	Corridas de IA simultáneas; más: 503 `AI_BUSY`.
`spring.webflux.multipart.max-disk-usage-per-part`	`20MB`	Tamaño máximo de un archivo subido.
`wasichai.files.max-bytes`	`64MB`	Techo absoluto de un archivo en el servidor.

5.5. Perfil `prod`

`application-prod.yml` cambia los valores por defecto a los de producción: base y secreto JWT obligatorios, sin usuario semilla, tokens de 4 h con revocación, límite de intentos de inicio de sesión, política de contraseñas, archivos en S3, sin valores de desarrollo para GeoServer, `sgspe-latex` ni el mapa base, encabezados del proxy (`server.forward-headers-strategy: framework`) y registros en JSON ECS. Lo que comprueba `ProdGuard` está en Sección 7.3.

Listado 1. Ejemplo de archivo de entorno de producción (sin secretos)

```
# /etc/sgspe/sgspe.env  — permisos 0600, propietario del servicio; los secretos los inyecta el gestor de secretos
SPRING_PROFILES_ACTIVE=prod
WASICHAI_DB_HOST=db.interno.ejemplo
WASICHAI_DB_PORT=5432
WASICHAI_DB_NAME=sgspe
WASICHAI_DB_USERNAME=sgspe_app
WASICHAI_DB_PASSWORD=${SECRETO_DB}              # desde el gestor de secretos
```

```
WASICHAI_JWT_SECRET=${SECRETO_JWT}              # 32 bytes aleatorios o más
WASICHAI_FILES_STORE=s3
WASICHAI_FILES_S3_BUCKET=sgspe-archivos
WASICHAI_FILES_S3_ENDPOINT=https://s3.region.ejemplo
SGSPE_DELIVERABLE_LATEX_URL=http://sgspe-latex:8080
SGSPE_MAP_BASE_STYLE_URL=https://mapas.ejemplo/estilo.json
WASICHAI_NOTIFICATIONS_EMAIL_ENABLED=true
WASICHAI_NOTIFICATIONS_EMAIL_FROM=avisos@cliente.ejemplo
SPRING_MAIL_HOST=smtp.interno.ejemplo
```

6. Instalación y arranque

Este capítulo instala un entorno de **desarrollo o demostración** en una sola máquina. Para producción, siga después Capítulo 7.

6.1. Requisitos

Herramienta	Versión	Para qué
JDK	25	Compilar y ejecutar el servidor (toolchain de Gradle). Gradle 9.8.0 llega con `./gradlew`.
PostgreSQL + PostGIS	18 + 3.6 en `compose.yml`	Base de datos. La demostración de este manual se verificó también con PostgreSQL 16 + PostGIS 3.4 (Sección 6.4).
Docker y Compose	reciente, local o remoto	Levantar PostGIS, MinIO, Mailpit, GeoServer y `sgspe-latex` con `compose.yml` (opcional si se instalan aparte).
Python	3.11+, solo biblioteca estándar	Scripts de `model/` (aprovisionamiento y datos) y de estos manuales.
Node.js y Yarn	Node 26+, Yarn 1	La interfaz `sgspe-ui` y las herramientas de formato del repositorio.
GDAL (`ogrinfo`, `ogr2ogr`)	3.8+	Opcional: importar y exportar Shapefile, KML, GeoPackage, FileGDB y GPX.
Acceso de lectura a GitHub Packages	token con `read:packages`	Descargar `wasichai:*` (Maven) y `@wasichai/*` (npm). Ver Sección 6.2.

Los requisitos de hardware no están fijados en el repositorio. Como referencia medida en la máquina donde se generaron estos manuales: la compilación usa hasta 3 GB de memoria (`org.gradle.jvmargs=-Xmx2g`, demonio de Kotlin `-Xmx3g`), el servidor arranca con la memoria por defecto de la JVM y la base de demostración completa ocupa menos de 200 MB.

6.2. Paquetes de wasichai

Las librerías de la plataforma se publican en GitHub Packages, que exige un token **incluso para leer**. El token nunca se escribe en un archivo del repositorio: se antepone a cada comando.

```
# en la raíz de sgspe
```

```
$ GITHUB_ACTOR=$(gh api user -q .login) GITHUB_TOKEN=$(gh auth token) ./gradlew build
```

Si falta el alcance, `gh auth refresh -s read:packages`. `settings.gradle.kts` declara además `mavenLocal()` como alternativa para el grupo `wasichai`: sin acceso a GitHub Packages, se puede compilar la plataforma desde su repositorio público y publicarla localmente.

```
# alternativa sin token: wasichai 1.3.0 desde el código fuente, en ~/.m2/repository
$ git clone https://github.com/wasichai/wasichai && cd wasichai
$ ./gradlew publishToMavenLocal -x test -x check
```

Para la interfaz, `sgspe-ui/.npmrc` apunta `@wasichai` a `npm.pkg.github.com`. La alternativa equivalente compila `wasichai-ui` (repositorio público) y empaqueta cada paquete con `yarn pack`; es la que se usó para las capturas de estos manuales y se describe en Sección 6.4.1.

6.3. Entorno de desarrollo con Docker (vía oficial)

Procedimiento

1. Prepare las variables (en la raíz de `sgspe`; `develop/.env` está en `.gitignore`):

   ```
   $ cp develop/example.env develop/.env
   $ set -a; source develop/.env; set +a
   ```

2. Levante los servicios de apoyo. El demonio de Docker del proyecto es remoto; si es su caso, abra antes el túnel SSH (`docs/develop/README.md` §3).

   ```
   $ docker compose up -d          # postgis, minio (+ minio-init), mailpit
   $ docker compose ps             # postgis y minio en «healthy»; minio-init termina con 0
   ```

3. Arranque el servidor. Las migraciones crean el esquema al arrancar:

   ```
   $ GITHUB_ACTOR=$(gh api user -q .login) GITHUB_TOKEN=$(gh auth token) ./gradlew bootRun
   $ curl -s http://localhost:8094/actuator/health/readiness
   {"status":"UP"}
   ```

4. Aprovisione la organización de desarrollo, en este orden (cada paso es idempotente):

   ```
   $ cd model
   $ python3 apply.py && python3 apply_roles.py && python3 apply.py --workflows
   $ python3 load_ubigeo.py && python3 load_indicators.py
   $ python3 publish_layers.py     # solo si hay GeoServer (WASICHAI_GIS_GEOSERVER_*)
   ```

5. Arranque la interfaz (en `sgspe-ui`):

```
$ yarn install && yarn dev      # http://localhost:5182, reenvía /api a :8094
```

Resultado esperado: en `http://localhost:5182/ login` se inicia sesión con `admin@wasichai.local` / `admin` (solo existe con `WASICHAI_SEED_DEV=true`) y el menú lateral muestra todas las secciones.

6.4. Entorno de demostración sin Docker (verificado)

Esta vía no necesita Docker ni túnel: usa un PostgreSQL con PostGIS instalado en la máquina y el jar del servidor. Es la que se usó para generar las capturas y la base de demostración de estos manuales, en Ubuntu 24.04.

Procedimiento

1. Instale PostgreSQL y PostGIS y cree un clúster en el puerto 5435 con un usuario que pueda crear bases:

```
$ sudo apt-get install -y postgresql-16 postgresql-16-postgis-3 openjdk-25-jdk-headless
$ sudo pg_createcluster 16 sgspe -p 5435 -- --encoding=UTF8 --locale=C.UTF-8
$ sudo pg_ctlcluster 16 sgspe start
$ sudo -u postgres psql -p 5435 -c "CREATE ROLE sgspe LOGIN CREATEDB PASSWORD 'sgspe'"
```

> `CREATE EXTENSION postgis` exige privilegios de superusuario o que la extensión sea de confianza en el servidor. Si el usuario `sgspe` no es superusuario, cree la extensión como `postgres` en cada base nueva, o dé al rol `SUPERUSER` **solo** en un clúster desechable de demostración.

2. Compile el jar del servidor (en la raíz de `sgspe`): `./gradlew bootJar` deja `build/libs/app.jar`.
3. Cree la base de demostración y cárguela con el script de estos manuales, que arranca el servidor, aprovisiona y carga el conjunto ficticio (detalle en Capítulo 10):

```
$ SGSPE_DEMO_PASSWORD='<12+ caracteres>' docs/manuals/scripts/seed-demo-data.sh --keep
-running
```

6.4.1. Interfaz con paquetes compilados localmente

Sin acceso a `npm.pkg.github.com`, la interfaz se puede instalar con los paquetes `@wasichai/ compilados del código fuente. Se hace en una *copia de trabajo` de `sgspe-ui` (nunca se versiona el cambio de `package.json`):

```
$ git clone https://github.com/wasichai/wasichai-ui && cd wasichai-ui
$ yarn install --frozen-lockfile && yarn build
$ for p in packages/*/; do (cd "$p" && yarn pack --filename ../../../tgz/wasichai-$(basename
"$p").tgz); done
# en la copia de sgspe-ui: quitar .npmrc y apuntar cada @wasichai/* a file:../tgz/wasichai-
<paquete>.tgz,
# también en "resolutions"; después:
$ yarn install && yarn build && WASICHAI_API_URL=http://localhost:8094 yarn preview
```

6.5. Comprobaciones después de instalar

Comprobación	Comando	Resultado esperado
Servidor vivo	`curl -s localhost:8094/actuator/health/liveness`	`{"status":"UP"}`
Servidor listo (base accesible)	`curl -s localhost:8094/actuator/health/readiness`	`{"status":"UP"}`
API de la plataforma	`curl -s localhost:8094/api/health`	`{"status":"UP",…}`
Modelo aplicado	`cd model && python3 apply_roles.py`	termina con `… 15 skipped`
Interfaz	`abrir http://localhost:5182/login`	formulario «Correo» / «Contraseña»

7. Despliegue y actualización

El repositorio no define el destino de producción (GAP-022): no hay manifiestos de Kubernetes, unidades de systemd ni imágenes de contenedor del servidor. Lo que sí define, y este capítulo documenta, es el **artefacto** que se despliega, la **configuración** de producción, las **comprobaciones** que impiden un arranque inseguro y el **procedimiento de actualización** del modelo de cada organización.

7.1. Artefactos

Artefacto	Cómo se obtiene
`sgspe-<versión>.jar`	Lo adjunta el flujo `publish.yml` a cada *release* de GitHub (`./gradlew bootJar -Pversion=<versión>`; el jar se llama siempre `build/libs/app.jar` y no hay jar plano). Se ejecuta con `java -jar`.
Estáticos de `sgspe-ui`	`yarn build` en `sgspe-ui` produce `dist/`, que sirve el proxy inverso con *fallback* de SPA.
Imagen `sgspe-latex`	Se construye desde `develop/latex/Dockerfile` (Debian fijado por *digest*, TeX Live, fuentes Source).
`model/`	El modelo, roles, workflows, automatizaciones e indicadores, que se aplican por REST a cada organización.

Las versiones las corta **release-please** (`release-please.yml`) a partir de los Conventional Commits: nunca se edita a mano `version=` en `gradle.properties`, `.release-please-manifest.json` ni `CHANGELOG.md`.

7.2. Topología de referencia

- Un proxy inverso con TLS sirve `sgspe-ui/dist` (incluido el *fallback* de `/admin/) y reenvía /api/ y /actuator/health/ al servidor. Debe *sobrescribir` (no agregar) `X-Forwarded-For` y `X-Forwarded-Proto`, porque el perfil `prod` confía en ellos para contar los intentos de inicio de sesión por dirección.
- El proxy envía la CSP (`yarn csp` en `sgspe-ui` la imprime) y aplica su propio límite de tasa al inicio de sesión.
- Una o más réplicas del servidor comparten la base; los trabajos y la cola de automatizaciones se coordinan con candados de PostgreSQL, así que pueden correr varias réplicas a la vez.
- PostgreSQL con PostGIS, almacenamiento S3, `sgspe-latex` y (opcional) GeoServer en una red interna, sin acceso desde Internet. `sgspe-latex` no necesita salida a Internet.

7.3. Comprobaciones de arranque (`ProdGuard`)

Con `SPRING_PROFILES_ACTIVE=prod`, `sgspe.prod.ProdGuard` se ejecuta antes de crear el contexto (y antes de las migraciones) y **aborta el arranque** si:

1. `WASICHAI_SEED_DEV=true`;
2. falta el secreto JWT, no se resolvió o es el de desarrollo;
3. la duración del token falta o es de 8 horas o más;
4. el almacenamiento de archivos no es `s3` o falta el bucket;
5. solo una de las dos claves S3 está definida, o alguna es la del MinIO de desarrollo;
6. el correo está encendido sin `SPRING_MAIL_HOST`, o el remitente está vacío o termina en `@sgspe.test`.

El mensaje de error nombra la regla incumplida. Las demás variables obligatorias del perfil (`WASICHAI_DB_*`) hacen fallar el arranque por falta de valor.

7.4. Primer despliegue

Procedimiento

1. Cree la base vacía con la extensión PostGIS y un usuario propietario para la aplicación.
2. Cree el bucket S3 y una identidad con permiso de lectura y escritura sobre él.
3. Configure el entorno del servicio (ejemplo en Capítulo 5) con los secretos desde el gestor de secretos.
4. Arranque el servidor con el perfil `prod`; las migraciones crean los esquemas `wasichai`, `app_data` y `sgspe`. Compruebe `GET /actuator/health/readiness` → `{"status":"UP"}`.
5. **Una sola vez**, siembre el permiso `MANAGE_TENANTS` para el rol `ADMIN` de la organización operadora (SQL en `docs/develop/README.md` §5, «Varios clientes»). Sin esa fila nadie puede crear organizaciones en producción.
6. Dé de alta cada organización cliente con `provision_tenant.py`, que crea la organización y ejecuta los pasos de aprovisionamiento como su administrador:

   ```
   # en sgspe/model; las contraseñas por variable o por consola, nunca como argumento
   $ SGSPE_OPERATOR_PASSWORD=… SGSPE_TENANT_ADMIN_PASSWORD=… python3 provision_tenant.py
   --core https://sgspe.ejemplo \
       --operator-email operador@… --name "Cliente Ejemplo" --slug cliente-ejemplo --admin
   -email admin@cliente.ejemplo
   ```

   Si un paso falla, corrija la causa y repita con `upgrade_tenants.py`: el alta de la organización no es idempotente, pero los pasos de aprovisionamiento sí.

7. Si hay GeoServer, cree el rol de solo lectura `sgspe_geoserver` (`docs/develop/README.md` §3) y publique las capas con `publish_layers.py`; si no, use `--skip-layers`.

7.5. Actualizar a una versión nueva

Procedimiento

1. Lea el `CHANGELOG.md` de la versión: indica cambios de modelo, de configuración o de comportamiento.
2. Haga un respaldo de la base y del almacenamiento (Capítulo 9) y anote la versión actual.
3. Despliegue el jar nuevo. Al arrancar, Flyway aplica solo las migraciones nuevas de cada módulo.
4. Suba el modelo de cada organización. `upgrade_tenants.py` repite los pasos de aprovisionamiento (todos idempotentes) en cada organización del inventario; una que falla no detiene a las demás:

```
# en sgspe/model; inventario con la forma de tenants.example.json, fuera del repositorio
$ SGSPE_TENANT_PASSWORD_CLIENTE_EJEMPLO=… python3 upgrade_tenants.py --core
https://sgspe.ejemplo \
    --inventory ~/tenants.json [--skip-layers]
```

5. Publique los estáticos nuevos de `sgspe-ui` (la interfaz y el servidor se versionan juntos).
6. Ejecute las comprobaciones posteriores (Sección 8.6).



`apply.py` sincroniza el modelo pero **nunca** renombra, cambia de tipo ni borra un campo en uso, y `apply.py --workflows` se detiene si un workflow nuevo quita un estado que tiene registros. Si un *release* requiere alguno de esos cambios, el `CHANGELOG.md` trae el procedimiento; no use `apply.py --drop`, que borra **todos** los datos.

7.6. Volver a la versión anterior

Las migraciones de Flyway no tienen vuelta atrás automática. Para revertir una actualización que ya migró la base, restaure el respaldo previo de la base y del almacenamiento (Capítulo 9) y despliegue el jar anterior. Por eso el respaldo previo a la actualización es obligatorio.

8. Operación y monitoreo

8.1. Arrancar, detener y reiniciar

El servidor es un proceso Java sin estado propio en disco (salvo `WASICHAI_FILES_LOCAL_PATH` con almacenamiento local). Se arranca con el entorno de Capítulo 5:

```
# desarrollo, desde la raíz de sgspe
$ ./gradlew bootRun
# cualquier entorno, con el jar
$ java -jar sgspe-<versión>.jar          # o build/libs/app.jar
```

- **Detener:** enviar `SIGTERM` (Ctrl+C o `kill <pid>`). Spring Boot termina las peticiones en curso y libera los candados de los trabajos; un trabajo interrumpido se retoma en su siguiente ciclo (JOB-05 retoma los borradores colgados tras `stale-after`, 15 min).
- **Reiniciar:** detener y arrancar. Con varias réplicas, reiniciar de a una: los trabajos los toma otra réplica.
- **Apagar solo los trabajos** (por ejemplo, durante una restauración o en una réplica de lectura): `SGSPE_JOBS_ENABLED=false`.

Los servicios de apoyo de desarrollo se controlan con Compose desde la raíz: `docker compose up -d`, `docker compose stop`, `docker compose ps`. `docker compose down -v` **borra los volúmenes y todos los datos**.

8.2. Comprobar el estado

Ruta (puerto 8094)	Qué indica
`GET /actuator/health/liveness`	El proceso responde. Si falla, reiniciar.
`GET /actuator/health/readiness`	El servidor puede atender (base accesible). Es la que debe usar el balanceador.
`GET /actuator/health`	Estado agregado. El SMTP no cuenta (`management.health.mail.enabled: false`).
`GET /api/health`	Salud de la plataforma: `{"status":"UP","application":…}`.

Las cuatro son públicas (no exigen token). No se configura la exposición de otros *endpoints* del actuator: rige la de Spring Boot por defecto.

Listado 2. Comprobaciones funcionales con un usuario

```
$ TOKEN=$(curl -s localhost:8094/api/auth/login -H 'Content-Type: application/json' \
    -d '{"email":"<correo>","password":"<contraseña>"}' | python3 -c 'import
json,sys;print(json.load(sys.stdin)["token"])')
$ curl -s -H "Authorization: Bearer $TOKEN" localhost:8094/api/auth/me            # quién
soy
$ curl -s -H "Authorization: Bearer $TOKEN" localhost:8094/api/sgspe/ai/status    # IA
disponible o no
```

8.3. Registros

El servidor escribe en la salida estándar: texto en desarrollo y **JSON ECS** con el perfil `prod`.

Logger o campo	Para qué
`sgspe.access`	Una línea por petición: `{método} {ruta} {estado} {ms}ms correlationId={id}`. No lleva la cadena de consulta ni el cuerpo. Una cancelación del cliente se registra como 499.
`correlationId`	El mismo valor que la cabecera `X-Correlation-Id` de la respuesta y el `correlation_id` de la auditoría. El cliente puede enviarlo (`^[A-Za-z0-9._-]{1,64}$`); si no, se genera un UUID.
`sgspe.*`	Avisos de los trabajos (por ejemplo, el año sin feriados configurados) y de las integraciones.
`com.embabel, org.springframework.ai`	En `WARN`: los registros nunca llevan preguntas, prompts ni respuestas de la IA.

Para seguir un error que reporta una persona, pídale la hora y la acción; el portal muestra el error, y el `X-Correlation-Id` de esa respuesta permite encontrar la línea de acceso y la fila de auditoría:

```
$ grep 'correlationId=<id>' servidor.log
$ psql -d sgspe -c "SELECT occurred_at, object_name, operation, user_id, reason FROM
wasichai.audit_log WHERE correlation_id = '<id>'"
```

8.4. Tareas periódicas del operador

Cuándo	Qué hacer
Diario	Revisar que el respaldo de la base y del almacenamiento terminó y es legible (Capítulo 9).

Cuándo	Qué hacer
Semanal	Revisar errores 5xx en las líneas `sgspe.access` y fallos de trabajos en el registro.
Mensual	Restaurar un respaldo en una base desechable y verificarlo (Sección 9.3).
Cada año	Agregar los feriados nacionales del año siguiente (Sección 8.5).
Cada versión	Seguir Capítulo 7 y las comprobaciones posteriores.

8.5. Feriados para los días hábiles

Los plazos de acuse y respuesta de las quejas cuentan días hábiles con los feriados de `sgspe.calendar.holidays` (`src/main/resources/application.yml`), que hoy trae 2026 y 2027. Para agregar un año:

1. Agregue la clave `'2028':` bajo `holidays` con una fecha `AAAA-MM-DD` por línea (solo feriados nacionales).
2. Ejecute `BusinessDaysTest` (valida todo año configurado) y despliegue.

Mientras un año falte, el servidor cuenta de lunes a viernes y lo advierte una vez por año en el registro. Una fecha de otro año o repetida impide arrancar. Los plazos ya calculados no se recalculan.

8.6. Comprobaciones después de un despliegue o una restauración

- ☐ `GET /actuator/health/readiness` responde `UP` en cada réplica.
- ☐ El registro no muestra errores de Flyway ni de `ProdGuard` al arrancar.
- ☐ Un usuario de cada perfil principal inicia sesión y ve sus proyectos.
- ☐ La lista de compromisos y la de QCS cargan y paginan.
- ☐ Una evidencia existente se descarga (prueba el almacenamiento de archivos).
- ☐ `apply_roles.py` contra la organización termina con `0 created, 0 updated`.
- ☐ Si hay GeoServer, el mapa territorial muestra las capas de referencia.
- ☐ Si hay `sgspe-latex`, un borrador de entregable llega a `LISTO`.

9. Copias de seguridad y restauración

La política de respaldo de producción (frecuencia, retención, cifrado, sitio) no está fijada en el repositorio (GAP-022): la define quien opera el despliegue. Este capítulo describe **qué** hay que respaldar para que una restauración sea completa y **cómo** hacerlo con las herramientas estándar de PostgreSQL, que son las que usan y prueban los scripts de la base de demostración (Capítulo 10).

9.1. Qué respaldar

Elemento	Dónde está	Cómo se respalda
Base de datos completa	PostgreSQL: esquemas `wasichai`, `app_data`, `sgspe` (todas las organizaciones comparten la base)	`pg_dump` lógico diario y, recomendado, archivo continuo de WAL para recuperación a un instante (PITR).
Archivos	Bucket S3 (`WASICHAI_FILES_S3_BUCKET`) o la carpeta `WASICHAI_FILES_LOCAL_PATH`	Versionado del bucket o copia sincronizada al mismo ritmo que la base.
Configuración	Variables de entorno del servicio y del proxy	En el gestor de secretos y la configuración del despliegue; nunca en el repositorio.
Modelo	`model/` en el repositorio, en la versión desplegada	Ya está versionado: basta anotar la versión.

> Base y archivos forman una unidad: un descriptor de `wasichai.stored_files` sin sus bytes da error al descargar, y unos bytes sin descriptor quedan huérfanos. Respalde ambos en la misma ventana y conserve juntos los de la misma fecha.

9.2. Respaldo lógico de la base

```
# formato personalizado: restauración selectiva y en paralelo con pg_restore
$ pg_dump --format=custom --compress=9 --no-owner --no-privileges -d sgspe -f sgspe-$(date
+%F).dump
# SQL plano y comprimido: legible, restaurable con psql
$ pg_dump --format=plain --no-owner --no-privileges -d sgspe | gzip -9 > sgspe-$(date +%F
).sql.gz
```

- Use un `pg_dump` de versión **igual o mayor** que la del servidor.
- `pg_dump` toma una instantánea coherente sin detener el servicio.

- El volcado incluye la extensión PostGIS, los tres esquemas, las tablas físicas de todas las organizaciones, las restricciones, las secuencias y la historia de Flyway. **No** incluye los roles de PostgreSQL (`pg_dumpall --roles-only` si se necesitan) ni los archivos.
- No se puede restaurar una sola organización por separado sin trabajo manual: sus tablas físicas llevan su id en el nombre y comparten las tablas de metadatos con las demás.

9.3. Restaurar



Restaure siempre en una base **nueva** y cambie el servidor a ella después de verificarla. Restaurar encima de la base en uso mezcla datos y no es reversible.

Procedimiento

1. Detenga el servidor o apague los trabajos (`SGSPE_JOBS_ENABLED=false`) para que nada escriba durante la operación.
2. Cree una base vacía con el mismo propietario y restaure:

```
$ createdb -T template0 -E UTF8 sgspe_restaurada
$ pg_restore --no-owner --no-privileges --exit-on-error --single-transaction -d
sgspe_restaurada sgspe-AAAA-MM-DD.dump
# o, con el SQL plano:
$ gunzip -c sgspe-AAAA-MM-DD.sql.gz | psql -X -v ON_ERROR_STOP=1 --single-transaction -d
sgspe_restaurada
```

3. Restaure los archivos de la misma fecha en el bucket o la carpeta que usará el servidor.
4. Verifique la base restaurada (conteos, restricciones, secuencias). `docs/manuals/scripts/demo_manifest.py` hace estas comprobaciones de forma genérica; para producción, guarde junto a cada respaldo los conteos de filas por tabla y compárelos.
5. Apunte el servidor a la base restaurada (`WASICHAI_DB_NAME`), arránquelo y ejecute Sección 8.6.
6. Conserve la base anterior hasta confirmar que la restaurada está bien; después archívela o bórrela.

No hace falta volver a aprovisionar: el modelo, los roles y los workflows de cada organización viajan dentro del volcado. Si el volcado es de una versión anterior del sistema, despliegue esa versión o ejecute después `upgrade_tenants.py` con la nueva.

9.4. Prueba periódica

Un respaldo que nunca se restauró no está probado. La base de demostración trae el ciclo completo automatizado y verificado (exportar, restaurar en otra base, comprobar conteos, claves foráneas, secuencias y geometrías, y arrancar el servidor contra ella): úselo como modelo para la prueba mensual de producción (Capítulo 10).

10. Base de demostración reproducible

La base de demostración permite a cualquier persona levantar un SGSPE con datos **ficticios** coherentes (los mismos de las capturas de los manuales), sin depender de una base preparada a mano. Se puede obtener de dos maneras, ambas verificadas:

- **Reconstruirla desde cero** con los mecanismos oficiales del proyecto (migraciones, aprovisionamiento y cargadores).
- **Restaurarla** desde el respaldo versionado en `docs/manuals/database/backups/`.

10.1. Qué contiene

Conjunto	Contenido
Oficial (`model/demo/*.json`, del proyecto)	El proyecto «DEMO-T1 · Proyecto Wayra Ejemplo» (minería, exploración) con sus partes, concesiones y campañas; 15 localidades, una comunidad y dos áreas de influencia aprobadas con 17 miembros; 85 grupos de interés (85 personas y 110 organizaciones) con 235 evaluaciones; 100 interacciones con 181 participantes y 2 visitas de campo; 12 compromisos en varios estados; 5 riesgos con 10 medidas y 20 revisiones; un plan de relacionamiento; el entregable «Diagnóstico social» con sus 35 secciones; 6 usuarios ficticios.
Ampliado (`data-base/seeds/ampliado/*.json`, de estos manuales)	Lo que el oficial no trae y los manuales necesitan mostrar: 5 quejas, 3 consultas y 3 solicitudes en todos los estados típicos; 2 aportes sociales con ítems (uno solicitado y uno aprobado) y un programa social; 2 incidentes; una alerta temprana; 2 tareas manuales; 5 usuarios ficticios más (QCS, inversión, aprobación, jefatura y auditoría) y sus asignaciones.
Evidencias (`scripts/seed-demo-evidence.py`)	4 archivos sintéticos generados por el script (2 PDF de una página y 2 PNG de un paisaje dibujado), vinculados a una queja, una consulta y un aporte.
Catálogos	Ubigeo del INEI (25 departamentos, 196 provincias, 1892 distritos) y 53 indicadores.

Todos los nombres de personas, organizaciones y lugares son inventados; los correos son `@sgspe.test`. Resultado medido (manifiesto del 2026-10-20): **3425 registros** de negocio, 169 tablas y 21 974 filas en total, 12 usuarios.

10.1.1. Usuarios de demostración

Correo	Rol
admin@wasichai.local	Administrador de la plataforma (contraseña de desarrollo `admin`; solo existe con `WASICHAI_SEED_DEV=true`)
gestion.social.demo@sgspe.test	Gerente social
actores.demo@sgspe.test	Gestor de grupos de interés
relacionista.demo@sgspe.test	Relacionista comunitario
campo.demo@sgspe.test	Agente de campo
compromisos.demo@sgspe.test	Gestor de compromisos
riesgos.demo@sgspe.test	Gestor de riesgos sociales
qcs.demo@sgspe.test	Gestor de quejas, consultas y solicitudes
inversion.demo@sgspe.test	Gestor de inversión social
aprobacion.demo@sgspe.test	Aprobador
jefatura.demo@sgspe.test	Jefe de proyecto
auditoria.demo@sgspe.test	Auditor

La contraseña de los usuarios `@sgspe.test` **nunca se escribe en un archivo**: llega por la variable `SGSPE_DEMO_PASSWORD` al cargar, y tras una restauración se fija con `reset-demo-passwords.py` (el respaldo solo trae el *hash* de la contraseña de quien lo generó, que nadie más conoce).



Estas credenciales son solo para entornos locales o desechables. Nunca cargue la demostración en una organización de producción: crea usuarios y datos ficticios que se mezclarían con los reales.

10.2. Requisitos

- PostgreSQL con PostGIS (probado con PostgreSQL 16.15 y PostGIS 3.4.2; la referencia del proyecto es 18 + 3.6) y un usuario que pueda crear bases y la extensión `postgis` (superusuario en un clúster desechable).
- JDK 25 y el jar del servidor (`./gradlew bootJar` → `build/libs/app.jar`).
- Python 3.11+, `psql`, `pg_dump`, `pg_restore`, `gzip`, `tar` y `curl`.

Los scripts leen la conexión de `SGSPE_DEMO_PGHOST` (localhost), `SGSPE_DEMO_PGPORT` (5435), `SGSPE_DEMO_PGUSER` (sgspe), `SGSPE_DEMO_PGPASSWORD` (sgspe) y `SGSPE_DEMO_DB` (sgspe_demo). Los archivos y registros locales van a `SGSPE_DEMO_STATE_DIR` (`~/.local/state/sgspe-demo`).

Salvaguardas

- Los scripts **solo** crean, borran o restauran bases cuyo nombre empieza por `sgspe_demo`, y solo en un servidor local (`localhost`, `127.0.0.1`, `::1`) salvo `SGSPE_DEMO_ALLOW_REMOTE=1`. Nunca tocan `sgspe` ni `sgspe_test`.
- La restauración siempre va a una base **nueva**; reemplazar una existente exige `--replace`.
- La exportación se niega si el servidor sigue conectado a la base (para que el volcado y el manifiesto describan el mismo estado), salvo `--allow-active`.

10.3. Reconstruir desde cero

```
# desde la raíz del repositorio
$ ./gradlew bootJar                                              # build/libs/app.jar
$ SGSPE_DEMO_PASSWORD='<12+ caracteres>' docs/manuals/scripts/seed-demo-data.sh --reset
```

`seed-demo-data.sh` hace, en orden: crea la base vacía con PostGIS (con `--reset` borra antes la anterior); arranca el servidor contra ella, cuyas migraciones crean el esquema; aplica `apply.py`, `apply_roles.py`, `apply.py --workflows`, `load_ubigeo.py` y `load_indicators.py`; carga el conjunto oficial con `load_demo.py`; carga el ampliado y las evidencias (salvo `--minimal`); y detiene el servidor (salvo `--keep-running`).

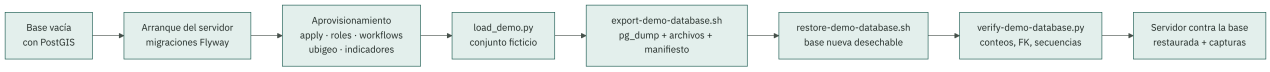


Figura 7. Ciclo de la base de demostración

Resultado medido (2026-10-20, servidor local): 1 min 28 s desde la base vacía. Salidas de cada paso: `apply.py` «done: 427 created, 29 updated, 0 skipped»; `apply_roles.py` «15 created»; workflows «19 workflows, 3 document types, 7 automations»; ubigeo «2113 created»; indicadores «53 created»; demostración oficial «896 created, 7 skipped, 266 commands, 3 warnings»; ampliado «26 created, 10 skipped»; evidencias «4 created».

Los 3 avisos de la carga oficial son esperados: tres indicadores (`KPI-ENG-05`, `KPI-ENG-08`, `KPI-GRV-07`) no se pueden cerrar en un periodo sin cifra completa («NO_SCALAR_VALUE»), y el cargador los informa sin detenerse.

Idempotencia: todo el proceso se puede repetir sin duplicar registros: cada cargador busca cada registro por su clave natural antes de crearlo. Medido: una segunda carga del conjunto ampliado termina con «0 created, 33 skipped», y una segunda corrida de las evidencias con «0 created, 4 skipped».

10.4. Exportar

```
$ docs/manuals/scripts/export-demo-database.sh          # con el servidor detenido
```

Artefacto (`database/backups/`)	Tamaño	Qué restaura
`sgspe-demo.sql.gz`	1,4 MB	Respaldo lógico **completo** en SQL plano: extensión, esquemas, tablas, índices, restricciones, historia de migraciones y los datos como `INSERT` con nombres de columna (de a 200 filas). Se restaura con `psql`.
`sgspe-demo.dump`	2,0 MB	El mismo contenido en formato personalizado de `pg_dump`, para `pg_restore` (restauración selectiva o en paralelo).
`sgspe-demo-files.tar.gz`	154 KB	Los archivos de wasichai-files (las 4 evidencias). Sin ellos, las descargas fallan.
`manifests/sgspe-demo.json`	—	El manifiesto: versión de SGSPE y de wasichai, *commit*, versiones de PostgreSQL, PostGIS y `pg_dump`, historia de migraciones por módulo, filas por tabla, restricciones, geometrías, registros por objeto, sumas SHA-256 de los artefactos y orden de restauración.



El SQL lleva datos, no solo `INSERT` sueltos: un volcado **de datos** no serviría sobre un esquema creado de nuevo, porque cada tabla física lleva en su nombre el id de la organización (ver el manual técnico, «Datos y base de datos»). Por eso ambos artefactos son volcados completos. La base usa identificadores UUID: no hay secuencias que ajustar.

10.5. Restaurar y verificar

```
# en una base nueva (por defecto sgspe_demo_restore); --format dump usa pg_restore
$ docs/manuals/scripts/restore-demo-database.sh [--db sgspe_demo_restore] [--format
sql|dump] [--replace]

# arrancar el servidor contra la base restaurada (no hace falta aprovisionar: el modelo
viaja en el volcado)
$ WASICHAI_DB_NAME=sgspe_demo_restore WASICHAI_FILES_STORE=local \
```

```
    WASICHAI_FILES_LOCAL_PATH=~/.local/state/sgspe-demo/files/sgspe_demo_restore java -jar
build/libs/app.jar

# fijar la contraseña de los usuarios ficticios y probar la API
$ SGSPE_DEMO_PASSWORD='<la suya>' python3 docs/manuals/scripts/reset-demo-passwords.py
$ SGSPE_DEMO_PASSWORD='<la suya>' PGHOST=localhost PGPORT=5435 PGUSER=sgspe PGPASSWORD=sgspe
\
    python3 docs/manuals/scripts/demo_manifest.py verify --db sgspe_demo_restore \
    --manifest docs/manuals/database/manifests/sgspe-demo.json --api http://localhost:8094 \
    --allow-growth wasichai.audit_log
```

El script comprueba las sumas SHA-256 antes de tocar nada, crea la base, carga el volcado en **una sola transacción** (si algo falla no queda nada a medias), extrae los archivos y verifica la base contra el manifiesto:

Comprobación	Resultado medido (SQL y *dump*)
Filas por tabla (169 tablas, 21 974 filas)	✓ idénticas
Secuencias	✓ (no hay: la base usa UUID)
Restricciones por tipo	✓ mismas cantidades
Restricciones sin validar	✓ solo las 2 del origen (las `CHECK` de `audit_log` que la migración V10 de wasichai crea `NOT VALID` a propósito)
Claves foráneas sin huérfanos	✓ 384 revisadas, 0 huérfanos
Geometrías no nulas e inválidas	✓ idénticas en 33 columnas
Historia de migraciones	✓ 13 módulos
Registros por objeto del modelo	✓ 3425
Archivos restaurados	✓ 4
API (con `--api`)	✓ salud, inicio de sesión, proyecto DEMO-T1 y 85 grupos de interés legibles

La carga del SQL tardó 2 s y la del *dump* 1 s. Después de usar la base, `audit_log` crece (por ejemplo, el cambio de contraseñas añade 11 filas): por eso la verificación posterior se corre con `--allow-growth wasichai.audit_log`.

Diferencias inevitables entre el original y el restaurado: ninguna en los datos. El servidor de prueba fue PostgreSQL 16 + PostGIS 3.4; restaurar en PostgreSQL 18 + PostGIS 3.6 es compatible hacia arriba, no hacia abajo.

10.6. Reiniciar el entorno

```
# borra y reconstruye la base de demostración desde cero (irreversible para esa base)
$ SGSPE_DEMO_PASSWORD='…' docs/manuals/scripts/seed-demo-data.sh --reset
# o vuelve al estado del respaldo
$ docs/manuals/scripts/restore-demo-database.sh --replace
```

10.7. Mantener los conjuntos

- Para validar los archivos sin servidor: `cd model && python3 load_demo.py --dir ../docs/manuals/database/seeds/ampliado --validate-only`.
- Al cambiar el modelo, vuelva a reconstruir, exportar y verificar; el manifiesto registra el *commit* con el que se generó cada respaldo.
- No agregue datos reales ni nombres de personas reales: el repositorio lo prohíbe.

11. Diagnóstico de fallos frecuentes

Empiece siempre por tres datos: la **hora**, la **acción** que se intentaba y el **`X-Correlation-Id`** de la respuesta (visible en las herramientas de desarrollo del navegador, pestaña Red). Con ellos encuentre la línea de acceso y, si hubo escritura, la fila de auditoría (Sección 8.3).

11.1. Compilación y arranque

Síntoma	Causa probable	Solución
`Could not find wasichai:wasichai-spring-boot-starter…`	Falta el token de GitHub Packages o su alcance `read:packages`.	Anteponer `GITHUB_ACTOR=… GITHUB_TOKEN=…` (Sección 6.2) o publicar wasichai en Maven local.
`Received status code 429 from server` al compilar	Maven Central limita las descargas.	Reintentar; Gradle reutiliza lo ya descargado.
`Connection refused` a `localhost:5435`	PostgreSQL no está arriba o falta el túnel SSH.	`docker compose ps`; abrir el túnel (`docs/develop/README.md` §3).
El arranque falla con un mensaje de `ProdGuard`	Configuración insegura con el perfil `prod`.	Corregir lo que nombra el mensaje (Sección 7.3).
Falta `wasichai.security.jwt.secret`	Perfil `prod` sin `WASICHAI_JWT_SECRET`.	Definir el secreto (32 bytes o más).
Puerto 8094 ocupado	Otro servidor corriendo.	`lsof -iTCP:8094 -sTCP:LISTEN` y detenerlo, o `SERVER_PORT`.
Error de Flyway al arrancar	Migración fallida o base de otra versión.	Leer el registro; no editar `flyway_history_*` a mano: restaurar el respaldo previo y revisar la versión.
`ERROR: permission denied to create extension "postgis"`	El usuario de la base no puede crear la extensión.	Crear la extensión como superusuario antes de arrancar.

11.2. Uso del portal

Mensaje en el portal	Significado	Qué hacer
«Correo o contraseña incorrectos»	Credenciales inválidas (401).	Comprobar el correo; en `prod`, tras 5 intentos fallidos se bloquea 15 min (429).

Mensaje en el portal	Significado	Qué hacer
«Su sesión venció»	El token expiró (8 h; 4 h en `prod`) o fue revocado.	Volver a escribir la contraseña en el diálogo.
«Su cuenta no tiene permiso para esta acción…»	403: falta el permiso del rol o la acción.	Revisar el rol en `model/roles.json` y la matriz de la Referencia técnica.
«El registro no existe o no es de sus proyectos.»	404: el registro está fuera de su alcance.	Comprobar la asignación vigente al proyecto (Equipo y parámetros).
«No está asignado a este proyecto: puede leerlo, pero no cambiarlo.»	Lee todos los proyectos pero no tiene asignación en este.	Pedir una asignación al administrador de proyectos.
«No se pudo: el registro está en otro estado.»	409: la transición ya no aplica.	Recargar la ficha.
«Otra persona cambió este registro…»	412: conflicto de `If-Match`.	«Recargar»; lo escrito se conserva.
«No hubo respuesta del servidor…»	Red caída o servidor detenido.	Comprobar `/actuator/health/readiness`.
«Sin mapa base»	Falta `SGSPE_MAP_BASE_STYLE_URL` (en `prod` no tiene valor por defecto).	Configurar el estilo del mapa base.
«El servicio de mapas de referencia no responde»	GeoServer apagado o inaccesible.	Revisar `WASICHAI_GIS_GEOSERVER_*` y el servicio.
Borrador del entregable en «FALLIDO»	`sgspe-latex` inaccesible (`PDF_CONVERTER_UNAVAILABLE`), mapas fallidos o datos incompletos.	Revisar `SGSPE_DELIVERABLE_LATEX_URL` y el registro de JOB-05; volver a pedir el borrador.
Importación con 422 `FORMAT_UNAVAILABLE`	Falta GDAL o el driver del formato.	Instalar GDAL 3.8+ o `SGSPE_GDAL_PATH`; XLSX, CSV y GeoJSON funcionan sin él.
«Preguntar» no responde (IA no disponible)	IA apagada para la organización o sin clave del proveedor (`GET /api/sgspe/ai/status` devuelve `enabled: false`).	`SGSPE_AI_ENABLED_ORGANIZATIONS` y `ANTHROPIC_API_KEY`.

11.3. Datos y modelo

Síntoma	Causa probable	Solución
`apply_roles.py` informa roles actualizados que nadie cambió	Alguien editó un rol en `/admin`.	Es lo esperado: `roles.json` manda. Llevar el cambio a `roles.json` si debe quedar.
`apply.py --workflows` termina con 1	Un workflow quita un estado que tiene registros.	Migrar esos registros primero; no forzar.
`load_demo.py`: «state X, expected Y»	Una carga anterior murió entre el alta de un registro y sus transiciones.	Corregir ese registro a mano o reconstruir la demostración con `seed-demo-data.sh --reset`.
Un estado «vencido» no se actualiza	JOB-01 corre cada hora (minuto 5) o los trabajos están apagados.	Esperar al ciclo o revisar `SGSPE_JOBS_ENABLED`.
Un año sin feriados en el registro	Falta el año en `sgspe.calendar.holidays`.	Sección 8.5.

12. Recuperación ante desastres

Este capítulo ordena las decisiones para volver a operar cuando se pierde un componente. Los objetivos de tiempo (RTO) y de pérdida de datos (RPO) no están definidos en el repositorio (GAP-022); los fija el responsable del servicio y determinan la frecuencia de los respaldos de Capítulo 9.

Pérdida	Impacto	Recuperación
Una réplica del servidor	Ninguno si hay otra réplica; si no, el servicio cae.	Arrancar otra instancia con la misma configuración. No guarda estado: los trabajos los retoma la réplica viva.
Base de datos	Servicio caído.	Restaurar el último respaldo en un servidor nuevo (Sección 9.3), con PITR si existe; luego los archivos de la misma fecha.
Almacenamiento de archivos	Las descargas de evidencias y entregables fallan; el resto funciona.	Restaurar el bucket desde su versionado o copia. Los archivos posteriores al respaldo se vuelven a subir desde su origen.
`sgspe-latex`	Los borradores de entregables quedan FALLIDO con `PDF_CONVERTER_UNAVAILABLE`.	Reconstruir la imagen desde `develop/latex/Dockerfile` y volver a pedir el borrador.
GeoServer	Sin capas de referencia ni mapas de informe; el proxy WMS responde 503.	Levantarlo y volver a ejecutar `publish_layers.py` (idempotente).
SMTP o proveedor de IA	Sin correos o sin IA; los avisos siguen en el portal.	Restablecer el servicio; nada que restaurar.
Secreto JWT comprometido	Tokens falsificables.	Cambiar `WASICHAI_JWT_SECRET` y reiniciar todas las réplicas: todas las sesiones se invalidan.
Credencial de base o S3 comprometida	Acceso no autorizado a los datos.	Rotar la credencial en el gestor de secretos, reiniciar y revisar la auditoría (`wasichai.audit_log`) por `correlation_id` y usuario.

12.1. Lista de comprobación de recuperación

1. Declarar el incidente, congelar cambios y anotar la hora del último dato bueno conocido.
2. Elegir el punto de restauración (respaldo o instante de PITR) y localizar los archivos de la misma fecha.
3. Restaurar en infraestructura nueva o limpia; nunca sobre la base dañada.
4. Arrancar el servidor con `SGSPE_JOBS_ENABLED=false`, verificar (Sección 8.6) y recién entonces encender los trabajos.
5. Comunicar a los usuarios qué intervalo de datos se perdió, para que lo vuelvan a registrar.
6. Documentar la causa y ajustar la frecuencia de respaldo si el RPO no se cumplió.

13. Pruebas y calidad

13.1. Niveles de prueba

Nivel	Comando (raíz de `sgspe`)	Qué cubre
Unitarias y estilo	`./gradlew build`	ktlint y las pruebas sin base de datos (excluye las etiquetas `integration`, `latex` y `slow`). Unos 300 archivos de prueba.
Integración	`./gradlew integrationTest --rerun`	Levantan la aplicación entera contra PostgreSQL + PostGIS (Testcontainers o `WASICHAI_TEST_DB_*`) y aprovisionan con los scripts reales de `model/`: API, guardas, workflows, matriz de permisos (15 roles × 90 objetos), automatizaciones y el conjunto de demostración. Unas 190 clases; de 30 minutos a más de una hora.
PDF con LuaLaTeX	`incluidas en integrationTest (etiqueta latex)`	Composición real con `sgspe-latex` (`SGSPE_DELIVERABLE_LATEX_URL`).
Lenta	`./gradlew slowTest --rerun`	A pedido, nunca en la CI: GeoServer y `sgspe-latex` reales; un T1 de unas 100 páginas.
Modelo	`cd model && python3 -m unittest -v`	Scripts de `model/` contra un core simulado y el modelo contra los documentos de diseño.
Trazabilidad	`python3 tools/check_traceability.py`	Una fila por requisito en `docs/requirements-traceability.md`.
Formato	`yarn format:check`	Prettier sobre YAML y JSON.
Interfaz	`yarn test, yarn e2e (en sgspe-ui)`	Vitest con jsdom y Playwright sobre la compilación de producción con una API simulada en el navegador.

Las pruebas no llaman nunca al proveedor de IA: `ANTHROPIC_API_KEY` y sus variantes quedan en blanco en `test` e `integrationTest`, y la única clase con clave la pone falsa y usa un modelo guionado.

13.2. Integración continua

`.github/workflows/ci.yml` corre en cada *push* a `main` y en cada *pull request*:

Trabajo	Pasos
`build`	JDK 25, GDAL, `./gradlew build`; sube los reportes si falla.
`integration`	GDAL, imagen `postgis/postgis:18-3.6`, construye y levanta `sgspe-latex` en 8097, `./gradlew integrationTest` (60 min de plazo).
`model`	`apply.py --validate-only`, `apply_roles.py --dry-run`, `publish_layers.py --validate-only`, `load_ubigeo.py --dry-run` y las pruebas de `model/`.
`traceability`	`tools/check_traceability.py` y sus pruebas.
`format`	`yarn format:check`.

Además: `commits.yml` valida los mensajes con commitlint (Conventional Commits, asunto en español) y el título del *pull request*; `release-please.yml` prepara las versiones; `publish.yml` adjunta el jar a cada *release*.

13.3. Verificaciones hechas para estos manuales

Lo que se ejecutó al preparar esta edición está registrado en `docs/manuals/CHANGELOG.adoc` y en el informe de la base de demostración (`database/manifests/sgspe-demo.json`). Lo que no pudo ejecutarse en el entorno de redacción (por ejemplo, la suite de integración completa, GeoServer o `sgspe-latex`) se indica allí como **no verificado**.

14. Mantenimiento de esta documentación

14.1. Estructura

Ruta (en `docs/manuals/`)	Contenido
`index.adoc, attributes.adoc`	Portada HTML y atributos comunes (producto, idioma, numeración, rutas).
`user/, administrator/, system/`	Los tres manuales: un archivo maestro (`manual-*.adoc`) que incluye sus capítulos.
`reference/`	La Referencia técnica: API, configuración y glosario escritos a mano, y `generated/` producido desde `model/`.
`operations/`	Instalación, despliegue, operación, respaldo, diagnóstico y recuperación (los incluye el manual técnico).
`database/`	README de la base de demostración, manifiesto (`manifests/`) y artefactos exportados (`backups/`).
`assets/`	Capturas (`screenshots/`), diagramas Mermaid (`diagrams/*.mmd` y su PNG/SVG) y marca (`branding/`).
`styles/`	Hoja de estilos HTML, tema PDF y fuentes IBM Plex (OFL) empaquetadas.
`scripts/`	Construcción, validación, capturas, diagramas, referencias y base de demostración.
`output/`	HTML y PDF generados (no se editan a mano).

La **fuente canónica** es AsciiDoc: el HTML y el PDF salen de los mismos archivos. Las secciones generadas llevan la marca `// GENERADO` y no se editan.

14.2. Herramientas

Herramienta	Versión	Instalación
Ruby + Asciidoctor, Asciidoctor PDF, Rouge, text-hyphen	las del `Gemfile.lock`	`cd docs/manuals && bundle install`
Node.js + Playwright + Mermaid	Node 22+; versiones de `package.json`	`cd docs/manuals && yarn install` (con `PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1` si ya hay Chromium, y `PLAYWRIGHT_CHROMIUM_EXECUTABLE` apuntándolo)

Herramienta	Versión	Instalación
Python	3.11+	Biblioteca estándar.
Poppler (`pdftoppm`, `pdffonts`, `pdfinfo`)	cualquiera	Opcional, para la validación visual del PDF.

14.3. Comandos

Comando (desde la raíz del repositorio)	Efecto
`docs/manuals/scripts/build-docs.sh`	Todo: referencias, diagramas, HTML y PDF.
`docs/manuals/scripts/build-docs.sh html | pdf`	Solo un formato.
`docs/manuals/scripts/build-docs.sh references`	Regenera `reference/generated/` desde `model/` y `docs/glossary.md`.
`docs/manuals/scripts/build-docs.sh diagrams`	Renderiza los diagramas que cambiaron.
`python3 docs/manuals/scripts/validate-docs.py`	Valida fuentes y salidas (Sección 14.5).
`node docs/manuals/scripts/generate-screenshots.mjs`	Vuelve a tomar las capturas contra el portal en marcha (Sección 14.4).
`docs/manuals/scripts/build-docs.sh clean`	Borra `output/`.

Los mismos comandos están en `docs/manuals/package.json` (`yarn docs:build`, `yarn docs:validate`, etc.).

14.4. Actualizar las capturas

Las capturas son de la aplicación real: el script inicia sesión con los usuarios ficticios de la base de demostración y recorre el portal a 1440 × 1000 píxeles.

Procedimiento

1. Levante la base de demostración y el servidor (`seed-demo-data.sh --keep-running`, o restaure el respaldo y arranque el servidor contra él).
2. Levante la interfaz en el puerto 5182 apuntando a ese servidor (`yarn preview` o `yarn dev` en `sgspe-ui`).

3. Ejecute las capturas:

```
$ cd docs/manuals
$ SGSPE_DEMO_PASSWORD='…' SGSPE_UI_URL=http://localhost:5182 node scripts/generate-screenshots.mjs
```

4. Revise las imágenes nuevas (`git diff --stat docs/manuals/assets/screenshots`) y reconstruya los manuales.

El script falla si una pantalla no carga o si aparece un mensaje de error, para no publicar capturas de estados transitorios. Los nombres de archivo son estables: una captura nueva reemplaza a la anterior en el mismo lugar.

14.5. Validación

`validate-docs.py` comprueba, y falla si algo no se cumple:

- que las referencias generadas estén al día con `model/`;
- que cada `include::` e `image::` apunte a un archivo que existe, y que no haya capturas sin usar;
- que no haya rutas absolutas locales, marcadores pendientes ni secretos aparentes (claves, tokens) en las fuentes;
- que el HTML generado no tenga enlaces internos rotos (anclas y archivos);
- que cada PDF exista, tenga páginas y lleve todas sus fuentes incrustadas.

14.6. Cuándo actualizar

Cambio en el sistema	Qué actualizar
`model/` (objetos, roles, workflows, indicadores)	`build-docs.sh references`; revisar los textos que nombran estados o roles.
Pantallas del portal	Capturas y el capítulo correspondiente del Manual de usuario.
Configuración o despliegue	Capítulo 5 y Capítulo 7.
Nueva versión	Basta reconstruir: la versión y la fecha se toman de `gradle.properties` y del día de construcción. Registrar los cambios en `docs/manuals/CHANGELOG.adoc`.